《AI 秘书-基于大模型的智能个人助理》

设计文档 (SDD)

Software Design Description

软件工程课程项目组

2026 年 4 月 24 日

摘要

本文档依据本小组项目在 GitHub 仓库 (`nis3366`) 的源代码提取并梳理而成。本文档深度剖析了基于 HarmonyOS / OpenHarmony (ArkUI & ArkTS) 构建的 “AI 秘书” 应用的内部架构。全文涵盖表现层组件的状态管理、业务逻辑层的大模型意图识别算法、安全风控拦截机制，以及底层 SQLite (RdbStore) 的读写分离事务设计。本文档不仅为开发团队的后续迭代提供了严格的代码级规范，同时也作为项目验收的核心技术交付物。

文档目录

1 引言 (Introduction) 3
1.1 编写目的 . . . 3
1.2 适用范围 . . . 3
1.3 术语表与缩略语 . . . 3

2 总体系统架构设计 (System Architecture) 3
2.1 设计理念与分层模型 . . . 3
2.2 工程目录结构映射 . . . 4

3 视图表现层详细设计 (View Layer) 4
3.1 会话中心 (`SessionSelect.ets`) . . . 4
3.2 核心聊天流页面 (`ChatDetail.ets`) . . . 4
3.3 沉浸式心理辅导 (`PsychChat.ets`) . . . 5

4 业务逻辑服务层详细设计 (Service Layer) 5
4.1 大模型通信与意图解析服务 (`LLMService.ets`) . . . 5
4.1.1 上下文组装机制 (Context Window Management) . . . 5
4.1.2 结构化 JSON 解析协议 . . . 6

5 安全风控模块设计 (`SecurityChecker.ets`) 6
5.1 拦截规则定义 . . . 6

6 数据持久化层设计 (Database & Executors) 6
6.1 物理表 Schema 详述 . . . 7
6.2 隔离机制：Read 与 Write 执行器 . . . 7
6.2.1 WriteExecutor (写操作与事务保障) . . . 7
6.2.2 ReadExecutor (读优化) . . . 8

7 错误处理与系统日志规范 8
7.1 错误码字典 (Error Codes) . . . 8
7.2 前端交互反馈 (`ActionDialog.ets`) . . . 8

8 总结与后续架构演进计划 8

1 引言 (Introduction)

1.1 编写目的

本文档旨在全面且细致地描述“AI 秘书”系统的软件架构设计与详细模块设计。通过定义类之间的关系、数据流向、UI 组件生命周期以及数据库表结构，指导开发人员进行无歧义的编码实现，并辅助测试人员设计精确的白盒测试用例。

1.2 适用范围

本文档适用于参与“AI 秘书”项目的所有后端开发工程师、前端 (ArkTS) 开发工程师、测试工程师及运维部署人员。

1.3 术语表与缩略语

tablehead **术语/缩写**	**全称**	**说明**
ArkTS	Ark TypeScript	HarmonyOS 优化的 TypeScript 扩展，主打声明式 UI 开发。
ArkUI	Ark User Interface	华为提供的声明式 UI 开发框架，包含完整的组件库。
LLM	Large Language Model	大语言模型（如盘古大模型或兼容的 OpenAI 接口模型）。
RDB	Relational Database	本项目中指代 OpenHarmony 的关系型数据库 API (SQLite)。
Prompt	System Prompt	预设给大模型的背景设定和指令，用于引导输出结构化数据。
Executor	Data Executor	数据执行器，采用设计模式对底层 SQL 操作进行业务层封装。

表 1: 术语与缩略语说明表

2 总体系统架构设计 (System Architecture)

2.1 设计理念与分层模型

本系统严格遵循 **MVVM (Model-View-ViewModel)** 与 **Clean Architecture** 理念，确保 UI 渲染与业务逻辑的极致解耦。系统整体划分为三大核心层级：

- **View Layer (视图层):** 由 `Pages` 和 `Components` 构成，单纯负责 UI 的声明式描述与用户手势事件捕获，不包含复杂的业务计算。
- **ViewModel / Service Layer (服务层):** 包含 `Utils` (如 `LLMService`) 和 `Security` (如 `SecurityChecker`)。负责处理来自 View 层的动作，发起网络请求，并执行输入输出的合规性校验。
- **Model / Data Layer (数据层):** 包含 `DatabaseManager` 和 `Executors`。全权负责本地数据的 CRUD 操作，对外暴露 Promise 异步接口。

2.2 工程目录结构映射

基于项目的 `entry/src/main/ets` 目录，工程结构布局如下：

```
entry/src/main/ets
├── components ..... 可复用的自定义 UI 组件
│   └── ActionDialog.ets ..... 全局弹窗组件
├── database ..... 数据库基础支持
│   └── DatabaseManager.ets ..... RDB 实例与建表逻辑
├── executors ..... 数据读写业务封装
│   ├── ReadExecutor.ets ..... 只读查询方法
│   └── WriteExecutor.ets ..... 落库与事务方法
├── pages ..... 系统路由级页面
│   ├── Index.ets ..... 启动页/鉴权路由
│   ├── Login.ets ..... 用户登录页
│   ├── SessionSelect.ets ..... 会话列表管理
│   ├── ChatDetail.ets ..... 核心交互区：常规对话/日程
│   └── PsychChat.ets ..... 核心交互区：心理辅导
├── security ..... 内容风控模块
│   └── SecurityChecker.ets ..... 敏感词与合规拦截
└── utils ..... 通用工具服务
    └── LLMService.ets ..... 大模型 HTTP 通信与意图解析
```

3 视图表现层详细设计 (View Layer)

视图层全量采用 ArkTS `@Component` 装饰器构建。本节详细剖析核心页面的状态流转与渲染机制。

3.1 会话中心 (`SessionSelect.ets`)

作为用户登录后的主枢纽，该页面展示所有历史对话列表。

- **状态驱动：** 声明了 `@State sessionList: Conversation[]`。当页面触发 `onPageShow()` 生命周期钩子时，调用 `ReadExecutor.getAllSessions()` 更新该状态，UI 引擎自动触发 `List` 组件的差分重绘。
- **交互逻辑：** 用户点击特定 `ListItem` 时，执行 `router.pushUrl({ url: 'pages/ChatDetail', params: { sessionId: item.id } })` 实现带参跳转。

3.2 核心聊天流页面 (`ChatDetail.ets`)

本页面的渲染性能和交互逻辑最为复杂。

```
@Entry
@Component
struct ChatDetail {
    // 路由传递的当前会话 ID
    @State sessionId: number = 0;
    // 双向绑定的输入框文本
    @State inputText: string = "";
    // 驱动列表渲染的消息数组
```

```
    @State messageList: MessageModel[] = [];
    // 控制加载动画
    @State isRequesting: boolean = false;
    // 列表滚动控制器，用于保持消息沉底
    private scroller: Scroller = new Scroller();

    async sendMessage() {
        if (!this.inputText.trim()) return;
        // 1. 安全校验 (SecurityChecker)
        // 2. 本地落库 user 消息 (WriteExecutor)
        // 3. 触发 LLM 请求 (LLMService)
        // 4. 解析结果并落库 assistant 消息及日程
    }
}
```

Listing 1: ChatDetail 核心状态变量定义截取

重绘优化策略：为了避免长列表导致的内存泄漏，聊天列表采用了 `LazyForEach` 或限制初次加载条数（如 20 条）并配合滚动事件进行增量加载。

3.3 沉浸式心理辅导 (`PsychChat.ets`)

继承了普通聊天的基础能力，但在表现上做了差异化处理：

- **样式重载：**通过引用 `resources/dark/element/color.json` 中的色值，将页面底色调整为舒缓的暖暗色调，以适应心理咨询的氛围。
- **功能精简：**隐藏了普通对话中“提取日程”相关的 UI 入口，确保用户的视觉焦点完全集中于文本沟通。

4 业务逻辑服务层详细设计 (Service Layer)

4.1 大模型通信与意图解析服务 (`LLMService.ets`)

该模块负责将自然语言转化为可被系统执行的机器指令，是“AI 秘书”智能化的核心。

4.1.1 上下文组装机制 (Context Window Management)

为确保多轮对话的连贯性，服务在发起 HTTP 请求前，会向数据库请求当前 `sessionId` 的最近 10 条历史消息。

```
export class LLMService {
    static async fetchResponse(currentMsg: string, history: Message[]): Promise<LLMResponse> {
        let systemPrompt = "你是一个智能助理，请识别用户意图并严格返回 JSON。";
        let messagesPayload = [
            { role: "system", content: systemPrompt }
        ];

        // 压入历史上下文
        history.forEach(msg => {
            messagesPayload.push({ role: msg.role, content: msg.content });
        });

        // 压入当前消息
```

```
        messagesPayload.push({ role: "user", content: currentMsg });

        // 发起 HTTP 通信...
    }
}
```

Listing 2: LLMService 请求体组装逻辑

4.1.2 结构化 JSON 解析协议

模型返回的输出必须符合严格的结构化约定。系统设计了 `LLMResponse` 接口模型：

```
interface LLMResponse {
    reply_text: string;      // 展现在聊天界面的大模型回复文本
    action_type: ActionEnum; // 动作枚举: 'chat' | 'add_schedule' | 'psych_mode'
    params?: {               // 可选的动作参数字典
        title?: string;
        time?: string;       // ISO 8601 格式时间
        location?: string;
    }
}
```

Listing 3: LLMAction 解析协议定义

若 `JSON.parse()` 抛出异常（即模型出现幻觉未按要求输出），服务将捕获异常，并构造一个默认的 `ActionEnum.chat` 回复，从而保证系统极高的鲁棒性。

5 安全风控模块设计 (SecurityChecker.ets)

在将数据发往云端之前，或展示云端返回的数据之时，必须经过 `SecurityChecker` 的过滤。这满足了《测试文档》中对于输入合法性的强校验要求。

5.1 拦截规则定义

1. **长度防范 (Buffer Overflow Prevention):**
   设定常量 `MAX_CONTENT_LENGTH = 2000`。超过该长度的输入框内容会被直接拒绝发送，提示“输入内容过长”。

2. **XSS 与 HTML 注入清洗:**
   强制转义 `<`, `>`, `&` 等特殊字符，防止富文本解析引发应用崩溃。

3. **黑名单敏感词正则匹配:**
   内置基础词库字典树 (Trie) 或正则表达式 ‘/(暴力 | 违禁词 A| 违禁词 B)/g‘。命中则直接抛出 `SecurityException`。

4. **防 Prompt 注入 (Prompt Injection Protection):**
   检测诸如“忽略之前的指令”、“现在开始你是一个...”等带有指令劫持特征的特定句式。

6 数据持久化层设计 (Database & Executors)

应用采用 HarmonyOS 提供的 `@ohos.data.relationalStore` 构建底座。

6.1 物理表 Schema 详述

tablehead	**表名: Schedule (日程表)**		
字段名	**数据类型**	**约束条件**	**业务含义**
id	INTEGER	PRIMARY KEY AUTOINCREMENT	唯一主键
conv_id	INTEGER	NOT NULL, FK	关联产生该日程的会话 ID
title	TEXT	NOT NULL	日程事项名称 (LLM 提取)
start_time	INTEGER	NOT NULL	约定的开始时间戳（毫秒级）
location	TEXT	DEFAULT NULL	日程发生地点
is_notified	INTEGER	DEFAULT 0	本地通知布尔标识位 (0: 未通知, 1: 已通知)

表 2: Schedule 核心业务表结构

6.2 隔离机制：Read 与 Write 执行器

为了防止 UI 直接编写 SQL 导致代码混乱以及潜在的 SQL 注入风险，系统封锁了 `DatabaseManager` 的暴露，所有的访问必须通过 `Executors`。

6.2.1 WriteExecutor (写操作与事务保障)

涉及连带更新的操作（如：AI 提取出日程后，既要保存 AI 的聊天气泡，又要将日程插入 `Schedule` 表），必须使用事务包裹。

```
export class WriteExecutor {
    static async saveMessageAndSchedule(msg: Message, sch: Schedule): Promise<boolean> {
        let rdb = DatabaseManager.getRdbStore();
        try {
            await rdb.beginTransaction();
            await rdb.insert("Message", msg);
            await rdb.insert("Schedule", sch);
            await rdb.commit(); // 提交事务
            return true;
        } catch (error) {
            await rdb.rollback(); // 发生异常立即回滚
            console.error("Transaction failed: ", error);
            return false;
        }
    }
}
```

Listing 4: WriteExecutor 事务处理伪代码

6.2.2 ReadExecutor (读优化)

包含 `getSchedulesByDate(date: string)` 和 `getMessagesBySession(id: number)` 等高频调用接口。采用基于索引的 `RdbPredicates` 提高检索速度，满足冷启动数据装载小于 `300ms` 的性能要求。

7 错误处理与系统日志规范

系统定义了统一的异常处理边界，确保应用在极端情况下不会闪退 (Crash)。

7.1 错误码字典 (Error Codes)

- **ERR_LLM_TIMEOUT (1001):** 模型推理超时（设定阈值 15000ms）。UI 层拦截此错误并弹出 `ActionDialog` 提示重试。
- **ERR_SEC_VIOLATION (2001):** 命中 `SecurityChecker` 黑名单。强制停止当前交互。
- **ERR_DB_LOCKED (3001):** 数据库并发写入冲突。触发 Exponential Backoff (指数退避) 重试机制。

7.2 前端交互反馈 (`ActionDialog.ets`)

使用 ArkUI 的 `CustomDialogController` 封装统一的报错/提示面板。不再依赖于简陋的 `Toast`，而是提供“查看详情”和“重新发起”的闭环交互体验。

8 总结与后续架构演进计划

当前代码库所实现的架构已非常成熟地支撑了“多会话上下文”、“基于意图识别的静默任务处理（提取日程）”以及“端侧高严格的内容安全防线”。

未来的技术演进方向：

1. **端云结合向量检索：** 引入 Vector Database 或端侧的轻量级向量索引，实现对用户历史长程记忆的 RAG (检索增强生成)，而不仅局限于最近 10 条的 Window。
2. **Tool-Use 接口扩展：** 在现有的 `ActionEnum` 框架下，`WriteExecutor` 可无缝扩展 `CallApi` 动作，通过接入外部天气、翻译、智能家居等 API，彻底转化为全能型 Agent。
3. **端侧小模型蒸馏：** 在后续版本中，探索将轻量级分类模型 (如 0.5B 参数) 直接部署于 OpenHarmony 端侧，用于基础意图的前置判定，从而降低云端 API 的 Token 消耗与网络延迟。